index.js & question.js — Concepts to Review

What went wrong across the app-wiring and question-rendering files

index.js

Default vs. named exports

`export default class Quiz` must be imported as `import Quiz from './quiz.js'` — no curly braces. Curly braces (`import {Quiz} from ...`) are only for named exports. Import paths also need to start with `./` for local files, or the browser looks for a package instead.

Importing the same name twice

The file imported `Question` twice, from two different (and wrong) paths. A module can only import a given name once — this is a hard syntax error, not just a style issue.

Calling a function instead of passing it as a callback

`addEventListener("click", resetToStart())` — the trailing `()` executes `resetToStart` immediately, right when the listener is being set up, and passes its return value (undefined) as the handler. Pass the function itself: `addEventListener("click", resetToStart)`.

CSS selector syntax: class vs. tag

`document.querySelector("retry-btn")` without a leading `.` searches for a literal `<retry-btn>` HTML tag, which doesn't exist. A class selector always needs the dot: `.retry-btn`.

const vs. let: reassignment

`const playerName = ''` followed later by `playerName = 'Player'` throws, because `const` bindings can never be reassigned after their initial value. Any variable that changes later in its life needs `let`.

Variable shadowing

A `let currentQuiz = null` was declared at the top of the file, but inside `startQuiz()` a new `const currentQuiz = new Quiz(...)` was declared. That creates a second, separate variable that only exists inside the function — the outer one never gets updated, so any other function reading the outer `currentQuiz` sees `null` forever.

Constructor parameter order

`new Quiz(playerName, category, difficulty, numberOfQuestions)` was called with a different order than the class actually declares: `constructor(category, difficulty, numberOfQuestions, playerName)`. JavaScript matches arguments by position, not by name, so every value landed in the wrong property.

Assigning to elements that don't have that property

`quizOptionsForm.value = ''` — a `<form>` element has no `.value` property, so this line silently does nothing. Similarly, `questionsContainer = ''` tries to overwrite a `const` DOM reference itself instead of clearing its content — it should be `questionsContainer.innerHTML = ''`.

A stray space breaking a regex

`/ ^(?:[1-9]|[1-4][0-9]|50)$/` has a leading space inside the slashes — that space becomes part of the pattern itself, so the regex basically never matches valid input. Regex literals are exact; nothing inside the slashes is 'whitespace to ignore.'

Re-registering a listener on every click

The Enter-key listener for the questions-number input was added *inside* the Start button's click handler. Every click on Start therefore adds one more copy of that listener, so after a few clicks, pressing Enter fires `startQuiz()` multiple times at once. Event listeners that should exist once belong outside handlers that can run repeatedly.

question.js

Calling a class method without `this`

`decodeHtml(...)` was called bare inside the constructor instead of `this.decodeHtml(...)`. Methods defined on a class aren't automatically in scope as standalone functions — they only exist on the instance, reached through `this`.

Using the wrong variable name inside a function

Inside `decodeHtml(html)`, the body referenced a variable called `text`, which was never defined anywhere — it should have used the actual parameter name, `html`.

Passing an array where a single value is expected

`incorrect_answers` from the API is an array of strings, but `decodeHtml` expects one string. It needs to be mapped over: `incorrect_answers.map(a => this.decodeHtml(a))`, decoding each entry individually.

Order of operations changes the math

`Math.round(this.index / total) * 100` rounds the fraction (almost always 0 or 1) before multiplying, so the result is only ever 0% or 100%. The multiplication has to happen *before* rounding: `Math.round((this.index / total) * 100)`.

Overwriting a value in a loop instead of building it up

`allAnswersStracture = `...`` inside a `for` loop replaces the whole string on every pass instead of adding to it, so only the last answer button ever survives. Accumulating inside a loop needs +=, not =.

Hardcoded content instead of dynamic data

The question text was a fixed string ('What is the capital of France?') instead of `${this.question}`, so every question screen showed the same text regardless of which question was actually loaded.

Registering a handler that fires immediately

`button.addEventListener("click", this.checkAnswer(button))` calls `checkAnswer` right away (during setup) and hands its return value to `addEventListener` as the handler — which is not a function, so nothing happens on an actual click. It needs to be wrapped so the call happens later: `() => this.checkAnswer(button)`.

Losing `this` inside a callback

`document.addEventListener("keydown", this.handleKeydown)` passes the method by reference, which detaches it from the instance — inside `handleKeydown`, `this` would then refer to `document`, not the Question instance. The method needs to be bound once (e.g. in the constructor: `this.handleKeydown = this.handleKeydown.bind(this)`) so both adding and later removing the listener refer to the exact same bound function.

Calling a variable as if it were a function

`this.timerInterval(() => setInterval(...))` tries to *call* `timerInterval`, which is just a property meant to *hold* the interval ID. It should be an assignment: `this.timerInterval = setInterval(() => {...}, 1000)`.

Logic written outside the loop it belongs in

All the per-second logic (updating the timer text, playing warning sounds, checking for time-up) was written directly inside `startTimer()`, outside of the `setInterval` callback — so it only ever ran once immediately, instead of once every second.

A decrement that cancels itself out

`this.timeRemaining = this.timeRemaining--` — post-decrement (`x--`) *returns the old value* before subtracting, and that old value then gets assigned right back, so the countdown never actually decreases. Just `this.timeRemaining--;` on its own line does the job.

Assigning to the result of a DOM lookup

`document.getElementById("time-up-warning") = timeUp` tries to overwrite the element reference itself, which isn't legal. The content goes on a property of that element: `document.getElementById("time-up-warning").innerHTML = timeUp`.

Operator precedence with `!` and comparisons

`!selectedAnswer.toLowerCase() === this.correctAnswer.toLowerCase()` — `!` binds tighter than `===`, so this negates the string first (turning it into `false`), then compares a boolean to a string, which is always false. The 'wrong answer' branch could never run. The fix is to negate the whole

comparison: `selectedAnswer.toLowerCase() !== this.correctAnswer.toLowerCase()`.

A condition that's always true — or never true

Two spots checked `if (button.dataset.answer)` or `if (!button.dataset.answer)` to decide which buttons to affect. Since every answer button always has a `data-answer` attribute, one of these conditions is always true and the other always false — meaning 'highlight the correct answer' ended up highlighting *every* button, and 'disable the other buttons' never disabled anything. The actual value needs to be compared, e.g. checking it against `this.correctAnswer`.

Instantiating from the wrong reference

`new this.question(...)` tries to use `this.question` as a constructor — but that property holds the decoded question *text*, not the `Question` class. It should be `new Question(...)`, using the imported class directly.

Regular functions vs. arrow functions and `this`

The restart button's handler was written as `function(){ this.onQuizEnd() }`. A regular `function` gets its own `this`, set by how it's called — for a click handler, that means `this` becomes the button element, not the Question instance, so `this.onQuizEnd` is undefined. Arrow functions don't have their own `this` — they inherit it from the surrounding code, which is what's needed here.